

Department of Computer Science, UNM

**CS361 Programming Project 2: Comparing Array vs Priority
Queue
Implementations of Dijkstra's Algorithm**

Spring 2026
Due: 5th May 2026

Team: 2 Students
100 Points

Contents

1	Project Overview	2
2	Learning Objectives	2
3	Implementations to Build	2
3.1	Version 1: Array-Based Dijkstra	2
3.2	Version 2: Priority Queue Dijkstra	2
4	Instructions	3
4.1	Step 1: Write the Introduction	3
4.2	Step 2: Implementation of Both Versions	3
4.3	Step 3: Test on Different Graph Types	3
4.4	Step 4: Record Experimental Results	3
4.5	Step 5: Analyze the Results	4
5	Examples of Sparse and Dense Graphs	4
5.1	Sparse Graph 1	4
5.2	Sparse Graph 2	4
5.3	Dense Graph 1	5
5.4	Dense Graph 2	6
6	Suggested Experimental Design	6
6.1	Input Sizes	6
6.2	Density Categories	6
6.3	Measurements	7
7	Analysis Section	7
7.1	Model Analysis Paragraph 1: Theoretical Expectation	7
7.2	Model Analysis Paragraph 2: Interpretation of Dense Graph Results	7
7.3	Model Analysis Paragraph 3: Interpretation of Sparse Graph Results	7
7.4	Model Analysis Paragraph 4: Theory vs Practice	7
7.5	Model Analysis Paragraph 5: Final Conclusion	8
7.5.1	Questions Students Should Answer in the Conclusion	8
8	Submission Checklist	8

1 Project Overview

Comparing Array vs Priority Queue Implementations of Dijkstra's Algorithm

The aim of this project is to study how the choice of data structure affects the performance of Dijkstra's shortest path algorithm. You will implement two versions of Dijkstra's algorithm and compare them theoretically and experimentally.

Core Question

How does the performance of Dijkstra's algorithm change when the next minimum-distance vertex is selected using:

- a simple array / linear scan, and
- a priority queue (min-heap)?

2 Learning Objectives

At the end of the project, you should be able to:

- explain the working principle of Dijkstra's algorithm,
- distinguish between graph representations such as adjacency list and adjacency matrix.
- analyze the time complexity of different implementations,
- design fair runtime experiments,
- interpret differences between theoretical and practical performance.

3 Implementations to Build

Implement the following two versions.

3.1 Version 1: Array-Based Dijkstra

In this implementation, the algorithm stores distances in an array and repeatedly scans all unvisited vertices to find the vertex with the smallest tentative distance.

Recommended representation:

- adjacency matrix for very dense graphs, or
- Improve with adjacency list on sparse graphs

Expected time complexity:

$$O(V^2)$$

where V is the number of vertices.

3.2 Version 2: Priority Queue Dijkstra

In this implementation, the next minimum-distance vertex is selected using a min-priority queue (usually a binary heap).

Recommended representation:

- adjacency list + min-heap.

Expected time complexity:

$$O((V + E) \log V)$$

where E is the number of edges.

4 Instructions

Please follow these steps.

4.1 Step 1: Write the Introduction

Write a short background section covering:

- what the shortest path problem is,
- when Dijkstra's algorithm is applicable,
- why non-negative edge weights are required,
- the idea of relaxation,
- the role of the data structure used for extracting the minimum-distance vertex.

4.2 Step 2: Implementation of Both Versions

For both implementations, you should:

- print the shortest distance from a source vertex to all other vertices,
- **optionally** reconstruct the shortest path using a parent array (extra credits +10 points).

4.3 Step 3: Test on Different Graph Types

You must implement both representations (Adjacency-list, matrix) for the array version. For heap Version, just implementing using Adjacency-list works fine.

- **Sparse graph:** number of edges is close to the number of vertices.
- **Dense graph:** number of edges is close to the maximum possible number of edges.

For an undirected graph, the maximum possible number of edges is:

$$\frac{V(V - 1)}{2}$$

4.4 Step 4: Record Experimental Results

For each input graph, students should record:

- number of vertices V ,
- number of edges E ,
- source vertex,
- runtime of array implementation,

- runtime of priority queue implementation,
- whether both implementations produced the same shortest path distances for all vertices.

4.5 Step 5: Analyze the Results

Compare the results with the theoretical expectations and explain any discrepancies.

5 Examples of Sparse and Dense Graphs

Below are four small example graphs that can be used for explanation, dry runs, or correctness testing.

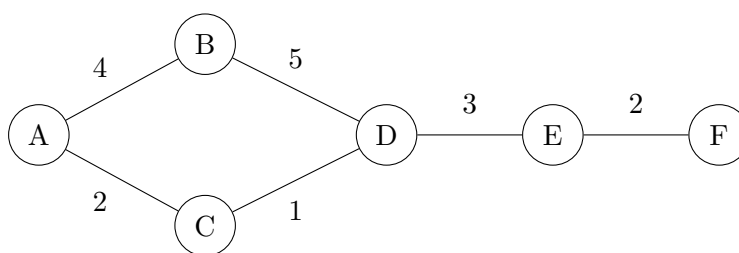
5.1 Sparse Graph 1

Vertices: $\{A, B, C, D, E, F\}$

Edges with weights:

- $(A, B, 4)$
- $(A, C, 2)$
- $(B, D, 5)$
- $(C, D, 1)$
- $(D, E, 3)$
- $(E, F, 2)$

This graph has 6 vertices and 6 edges, which is sparse because the number of edges is close to the number of vertices.



5.2 Sparse Graph 2

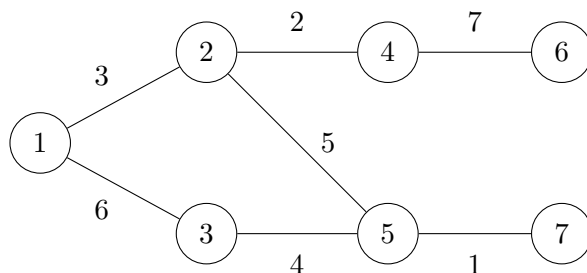
Vertices: $\{1, 2, 3, 4, 5, 6, 7\}$

Edges with weights:

- $(1, 2, 3)$
- $(1, 3, 6)$
- $(2, 4, 2)$
- $(3, 5, 4)$
- $(4, 6, 7)$

- (5, 7, 1)
- (2, 5, 5)

This graph has 7 vertices and 7 edges, again making it sparse.



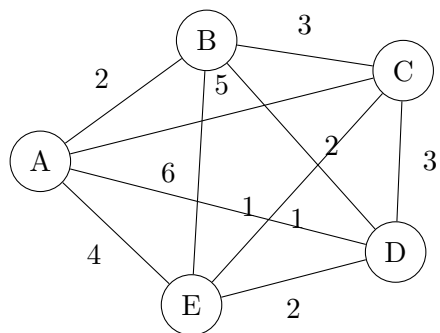
5.3 Dense Graph 1

Vertices: $\{A, B, C, D, E\}$

Edges with weights:

- (A, B, 2)
- (A, C, 5)
- (A, D, 1)
- (A, E, 4)
- (B, C, 3)
- (B, D, 2)
- (B, E, 6)
- (C, D, 3)
- (C, E, 1)
- (D, E, 2)

This graph is dense because with 5 vertices, the maximum number of undirected edges is $5 \cdot 4/2 = 10$, and all 10 possible edges are present.



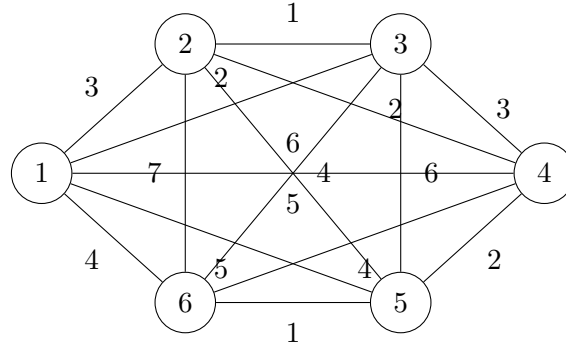
5.4 Dense Graph 2

Vertices: $\{1, 2, 3, 4, 5, 6\}$

Edges with weights:

- (1, 2, 3) (1, 3, 2) (1, 4, 6)
- (1, 5, 5) (1, 6, 4)
- (2, 3, 1) (2, 4, 2) (2, 5, 4) (2, 6, 7)
- (3, 4, 3) (3, 5, 6) (3, 6, 5)
- (4, 5, 2) (4, 6, 4)
- (5, 6, 1)

This graph has 6 vertices and 15 edges. Since the maximum number of undirected edges for 6 vertices is $6 \cdot 5/2 = 15$, this is also a fully connected dense graph.



6 Suggested Experimental Design

You should use the provided example graphs (2 sparse and 2 dense). In addition, you must generate at least one additional dense graph for your experiments.

6.1 Input Sizes

Students can test graph sizes such as:

$$V \in \{100, 500, 1000, 2000, 5000\}$$

with different edge densities.

6.2 Density Categories

A practical classification is:

- **Sparse:** $E \approx V$, $2V$, or $3V$
- **Medium density:** $E \approx V\sqrt{V}$
- **Dense:** $E \approx 0.5 \cdot \frac{V(V-1)}{2}$ or higher

6.3 Measurements

For each graph instance, you should measure:

- average runtime over five trials,
- memory usage,
- correctness of output distances

Example Table:

Graph #	Graph Type	Method	Time (ms)	Memory (MB)
1	Sparse-1	Array (Adjacency List)		
1	Sparse-1	Heap (Adjacency List)		
2	Sparse-2	Array (Adjacency List)		
2	Sparse-2	Heap (Adjacency List)		
3	Dense-1	Array (Adjacency Matrix)		
3	Dense-1	Heap (Adjacency List)		
4	Dense-2	Array (Adjacency Matrix)		
4	Dense-2	Heap (Adjacency List)		
5	Dense-3	Array (Adjacency Matrix)		
5	Dense-3	Heap (Adjacency List)		

Table 1: Experimental Comparison of Dijkstra Implementations

7 Analysis Section

7.1 Model Analysis Paragraph 1: Theoretical Expectation

Discuss Time complexity of Dijkstra's algorithm with array vs heap. Also give valid explanations.

7.2 Model Analysis Paragraph 2: Interpretation of Dense Graph Results

Discuss your interpretation of Dense Graph Results, time, space and possible reasons.

7.3 Model Analysis Paragraph 3: Interpretation of Sparse Graph Results

Discuss your interpretation of Sparse Graph Results, time, space and possible reasons.

7.4 Model Analysis Paragraph 4: Theory vs Practice

The experimental results may not exactly match the asymptotic analysis for small graph sizes. This happens because theoretical complexity focuses on long-term growth, while actual runtime is also affected by implementation details, machine speed, compiler optimizations, cache behavior, and constant factors. For small inputs, these factors may hide the theoretical advantage of the priority queue version. For larger inputs, the theoretical trend should become more apparent. Depending on your results, write your analysis here in short.

7.5 Model Analysis Paragraph 5: Final Conclusion

Your Final Thoughts.... and ...

7.5.1 Questions Students Should Answer in the Conclusion

Students should include a discussion section answering questions such as:

1. Did both implementations always produce the same shortest path distances?
2. On which graph types was the priority queue implementation clearly faster?
3. Was there any graph size where the array implementation was still competitive?
4. How closely did the measurements follow the theoretical complexity?
5. What practical factors may explain unexpected results?

8 Submission Checklist

Before submission, students should verify that they have included:

- source code for both implementations,
- at least two sparse and three dense example graphs,
- runtime, space comparison tables,
- plots or charts if possible (optional),
- a theory section,
- an analysis section,
- a final conclusion based on evidence.